

# Change Management and Database Performance Thresholds

|                                                    |                                                                 |                                                  |
|----------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------|
| <b>Author:</b> Database Administration (DBA) Group | <b>Target Audience:</b> Database Developers / Backend Engineers | <b>Compliance Target:</b> High-Availability SLAs |
|----------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------|

## 1.0 Purpose & Infrastructure Protection

Database deadlocks and unoptimized data access architectures directly cause system-wide transaction degradation, leading to customer churn and inflated database processing costs. This procedure dictates strict design constraints for relational engine transactions and queries to ensure platform stability during peak transaction windows.

## 2.0 Transactional Controls & Optimization Rules

### Section 4.1 Transaction Scoping and Termination Rules

All multi-table write operations inside the relational database engine must be tightly scoped and explicitly managed. Every transaction sequence must be bound within structured try-catch execution frameworks. A transaction must never be left uncommitted or dangling while awaiting an external software callback. It is mandatory that an explicit COMMIT TRANSACTION or a definitive ROLLBACK TRANSACTION is executed across all possible code logic termination paths.

### Section 4.4 Mandatory Indexing Strategies

Any relational database query involved in transactional join logic must execute against verified covering or clustered indexes. Performing queries that trigger full table scans on high-throughput columns (e.g., MerchantId, Status, SKU) inside production loops is completely prohibited. Failure to provide appropriate indexing strategies causes lock escalation and blocks concurrent user updates.

**OPERATIONAL IMPACT:** Uncommitted transactions hold exclusive locks (\$X\$) indefinitely. This locks down tables, starves the engine's connection pool, pushes virtual machine CPU metrics to 100%, and results in platform wide outages that break customer SLAs.